

Layered Network Programming with SNode.C

Building Multi-Protocol Applications in Modern C++

Volker Christian

Contents

I	Book Proposal Package	1
1	Working title	2
1.1	Alternative titles	2
2	Proposal summary	3
3	One-sentence positioning	4
4	Back-cover description	5
5	Manuscript status	6
6	Source-version baseline	7
7	Audience	8
7.1	Not the audience	8
8	Reader promise	9
9	Why this book should exist	10
10	Author and framework relationship	11
11	Author background	12
12	What makes this book different	13
13	Pedagogical approach	14
14	Chapter plan	15
14.1	Part I — Getting Oriented	15
14.2	Part II — The SNode.C Architecture	15
14.3	Part III — Networking Foundations in SNode.C	15
14.4	Part IV — From Raw Connections to Application Protocols	15
14.5	Part V — Configuration and Operational Behavior	15
14.6	Part VI — Secure and Robust Communication	15
14.7	Part VII — Web Protocols and Web Applications	16
14.8	Part VIII — IoT and Message-Oriented Systems	16
14.9	Part IX — Persistence and Full Systems	16
14.10	Part X — Building, Porting, and Maintaining	16
14.11	Part XI — Designing with SNode.C	16
14.12	Part XII — Building a MiniGateway Application	16
14.13	Epilogue	16
15	Companion source material	17
16	Verification status	18
17	Evidence and adoption status	19

18 Commercial / academic fit	20
19 Principal acquisition risks	21
19.1 Risk 1: Capped market	21
19.2 Risk 2: Manuscript length	21
19.3 Risk 3: First-party perspective	21
19.4 Risk 4: Verification status	21
19.5 Risk 5: Adoption evidence	21
20 Risk mitigation already present in the package	22
21 Best acquisition argument	23
22 Recommended publisher handling	24
 II Evidence Sheet: SNode.C Book Proposal	 26
23 Purpose of this sheet	27
24 Public project evidence	28
24.1 SNode.C repository	28
24.2 MQTTSuite repository	28
24.3 Related public repositories	28
25 Applied-system evidence	29
25.1 MQTTSuite as reference ecosystem	29
25.2 MiniGateway as guided capstone	29
25.3 Companion examples	29
26 Teaching-context evidence	30
27 Educational and interdisciplinary technical use cases	31
28 Author-confirmed professional context	32
29 Verification evidence inside the package	33
30 What is not yet claimed	34
31 Evidence gaps to close for a stronger acquisition case	35
32 Current evidence verdict	36

Part I

Book Proposal Package

Chapter 1

Working title

Layered Network Programming with SNode.C
Building Multi-Protocol Applications in Modern C++

1.1 Alternative titles

- SNode.C: Layered Network Programming in Modern C++
- The SNode.C Guide: Building Multi-Protocol Applications in Modern C++
- Designing Network Applications with SNode.C

Chapter 2

Proposal summary

This is a complete, architecture-first technical book about SNode.C, a layered, event-driven C++ framework for network applications. It is written as a first-party guide: the author is the creator and maintainer of the framework and explains the design from inside the system. The manuscript teaches SNode.C as a coherent model for building multi-protocol applications, not as a generic survey of C++ networking libraries.

The book is best positioned as a specialist professional title, course companion, or project-authoritative guide for experienced C++ developers and technical system builders. It also supports teachers, advanced students, makers and prototypers, and technically involved domain teams working with layered network software, IoT communication, embedded Linux, networked data collection, or multi-protocol application design. It should not be positioned as a broad Boost.Asio competitor, a beginner C++ book, a nontechnical science handbook, or a general socket-programming survey.

Chapter 3

One-sentence positioning

An architecture-first guide to SNode.C for mature technical readers who want to build, teach, prototype, or supervise layered, event-driven, multi-protocol network applications with explicit boundaries between endpoint identity, transport, connection handling, protocol behavior, configuration, diagnostics, deployment, persistence, and application roles.

Chapter 4

Back-cover description

SNode.C is a modern C++ framework for event-driven, layered network applications. This book teaches the framework as an architectural system: lower communication families, stream transport, connection objects, protocol contexts, configuration, diagnostics, deployment, persistence, and complete application roles are shown as parts of one design.

The book starts with small working examples and builds toward HTTP, Server-Sent Events, WebSocket, MQTT, MQTT over WebSocket, database-backed application state, Linux/OpenWrt deployment, testing, and system-level design. The final part develops MiniGateway, a guided application that combines HTTP status routes, SSE observation, MQTT publication, application-owned state, and an optional Unix-domain input boundary.

This is not a generic networking textbook and not a black-box recipe book. It is a practical architecture guide for readers who want to understand how a real C++ network framework is structured, how its layers interact, and how to extend applications without losing the boundaries that make the system understandable.

Chapter 5

Manuscript status

The manuscript is complete in draft form. It contains:

- front matter: preface, reading guidance, conventions, author/framework disclosure, acknowledgements;
- 38 chapters in 12 technical parts;
- an epilogue;
- further reading;
- a LaTeX-generated technical index;
- figures and diagrams;
- companion example source trees;
- MiniGateway base and extended source trees;
- source-version and verification notes.

The package uses a public SNode.C source baseline: release tag `v1.0.2`, commit `6e475262084ae2dab2daef8781ab9e4adb82d18e`. The manuscript is not presented as a description of arbitrary future repository state.

Chapter 6

Source-version baseline

The manuscript is aligned with the source baseline recorded in `SOURCE-VERSION.md`: repository `SNodeC/snode.c`, public release tag `v1.0.2`, commit `6e475262084ae2dab2daef8781ab9e4adb82d18e`. This matters because `SNode.C` is an active framework and public branches may move after a manuscript package is produced.

The reader-facing workflow is to check out the public tag. The full commit SHA remains the authoritative pin for reproducibility and review.

Chapter 7

Audience

The center of the audience is the experienced C++ developer or technical system builder who needs to understand how SNode.C applications are structured, extended, diagnosed, configured, and deployed. This reader is expected to read C++ code, reason about framework boundaries, and care about long-lived maintainability rather than only about the shortest path to a demo.

The most important adjacent channel is teaching. The book has a course-friendly shape: small examples, visible architecture, protocol layering, operational behavior, deployment, and a MiniGateway capstone. Teachers and lecturers can use it as a spine for advanced courses or laboratories in network programming, IoT systems, embedded Linux, media technology, or systems-oriented software design.

Advanced students are a natural audience when they already know the broad idea of protocol layers but need to see those layers expressed in real C++ code. Makers and prototypers are also plausible readers when their device, dashboard, gateway, router, sensor, home-lab, artistic, or experimental systems have grown beyond recipe-level composition.

The book can also support technically involved scientists, domain researchers, and interdisciplinary teams who work with data-collection and integration systems alongside developers. That audience is addressed through architectural relevance—sensing, transport, storage, observation, control, and management surfaces—not by turning the book into a nontechnical science handbook.

7.1 Not the audience

This book is not primarily for readers looking for a general survey of C++ networking libraries, a Boost.Asio tutorial, a beginner C++ book, a general TLS/HTTP/MQTT reference, an introductory IoT cookbook, or a nontechnical handbook for scientific measurement projects.

Chapter 8

Reader promise

After reading this book, the reader will be able to:

- build clients and servers with SNode.C;
- understand the framework’s layer model and runtime vocabulary;
- choose among IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, and Bluetooth L2CAP;
- configure applications in code, on the command line, and with configuration files;
- use TLS within the SNode.C connection model;
- work productively with HTTP, Express-like routing, SSE, WebSocket, MQTT, and MQTT over WebSocket;
- structure applications around configured roles, handles, runtime-visible instances, connections, `SocketContextFactory`, and `SocketContext`;
- reason about timeouts, retries, reconnects, logging, diagnostics, and backpressure;
- understand CMake components, public headers, linking, Linux/OpenWrt deployment, and testing shape;
- extend SNode.C applications without damaging architectural boundaries;
- reason about sensing, transport, storage, observation, control, and management surfaces in networked data-collection systems.

Chapter 9

Why this book should exist

Many networking books focus either on raw sockets or on one application protocol. Many web books abstract away transport and connection structure. Many IoT and data-collection books stay at solution level and never explain the system underneath.

SNode.C is useful as a book subject because it connects those layers without hiding them. It gives readers a concrete framework in which endpoint identity, transport form, connection lifetime, protocol meaning, configuration, diagnostics, deployment, application state, and management surfaces can be explained together.

The book therefore has two purposes:

1. to serve as the official architecture-first guide to SNode.C;
2. to act as a worked study of layered, event-driven network architecture in modern C++.

Chapter 10

Author and framework relationship

This is a first-party book. The author is the creator and maintainer of SNode.C and writes from inside the design of the framework. That is a strength when the goal is to explain intent, vocabulary, source layout, tradeoffs, and extension boundaries. It is also a limit: the book is not an independent comparative review of all C++ networking frameworks.

The manuscript addresses this directly in the front matter. It should be evaluated as a practical guide to SNode.C and as a concrete architectural case study, not as a market-neutral survey.

Chapter 11

Author background

Volker Christian is a software developer, educator, and systems architect focused on modern C++, event-driven network software, IoT communication, media technology, and layered protocol design. He is the creator and lead developer of SNode.C and the related MQTTSuite applications.

His author-confirmed professional background includes theoretical physics studies in Graz, work at Ars Electronica Futurelab, four years as a university assistant at Johannes Kepler University Linz in the Pervasive Computing group, work with ORF Kunstradio, participation around documenta X in Kassel, and many media-technology projects connected to Ars Electronica Futurelab. The public evidence sheet separates such author-supplied CV context from externally verifiable repository evidence.

Chapter 12

What makes this book different

1. **Architecture first, APIs second**

The book teaches SNode.C as a coherent design rather than as a catalog of calls.

2. **Lower and higher layers in one story**

IPv4, IPv6, Unix domain sockets, Bluetooth RFCOMM, Bluetooth L2CAP, TLS, HTTP, SSE, WebSocket, Express-like routing, MQTT, MQTT over WebSocket, MariaDB, Linux/OpenWrt deployment, and MiniGateway are presented as parts of one layered model.

3. **Framework guide plus architectural case study**

The reader learns SNode.C, but also sees a complete example of how a C++ network framework can organize roles, handles, connections, contexts, factories, configuration, and diagnostics.

4. **Small examples leading to system thinking**

The book starts with an echo pair and gradually reaches multi-protocol system design.

5. **Complete guided application**

MiniGateway gives the book a concrete capstone: a base version and an extended version that preserve existing protocol surfaces while adding another input boundary.

Chapter 13

Pedagogical approach

The book uses recurring teaching patterns:

- mental model before API detail;
- minimal example before larger system;
- explicit layer mapping;
- source-level orientation without becoming a line-by-line reference;
- safe extension framed around preserving boundaries;
- chapter takeaways for durable concepts;
- a technical index for lookup after first reading.

Chapter 14

Chapter plan

14.1 Part I — Getting Oriented

1. Why SNode.C Exists
2. Preparing Your Environment
3. Your First Working Program: The Echo Pair
4. Reading the Codebase with Confidence

14.2 Part II — The SNode.C Architecture

5. The Mental Model of SNode.C
6. Core Runtime and Event Processing
7. Layers in Practice: Network, Transport, Connection, Application

14.3 Part III — Networking Foundations in SNode.C

8. Socket Addresses and Address Semantics
9. Servers, Clients, and Connections
10. IPv4 and IPv6 as the Primary Teaching Path
11. Unix Domain Sockets
12. Bluetooth in SNode.C: RFCOMM and L2CAP

14.4 Part IV — From Raw Connections to Application Protocols

13. Writing `SocketContext` Classes Well
14. Writing `SocketContextFactory` Classes Well
15. Building the Same Protocol over Different Lower Layers

14.5 Part V — Configuration and Operational Behavior

16. Configuration Philosophy in SNode.C
17. Application and Instance Configuration in Detail
18. Logging, Diagnostics, and Runtime Introspection

14.6 Part VI — Secure and Robust Communication

19. TLS Across the Framework
20. Timeouts, Retries, and Failure Modes

14.7 Part VII — Web Protocols and Web Applications

- 21. The HTTP Layer
- 22. The Express-Like Framework
- 23. Server-Sent Events and Real-Time HTTP
- 24. WebSocket and Protocol Upgrade

14.8 Part VIII — IoT and Message-Oriented Systems

- 25. MQTT Support in SNode.C
- 26. MQTT over WebSocket
- 27. Designing IoT Systems with Multiple Protocols

14.9 Part IX — Persistence and Full Systems

- 28. Database Support and Application State
- 29. Learning from the Applications in `src/apps`
- 30. From Applications to Systems
- 31. MQTTSuite as a Reference Ecosystem

14.10 Part X — Building, Porting, and Maintaining

- 32. CMake Components and Linking Strategy
- 33. Deployment on Linux and OpenWrt
- 34. Testing, Debugging, and Benchmarking

14.11 Part XI — Designing with SNode.C

- 35. Architectural Judgment: Choosing the Right Layer and Boundary
- 36. Extending the Framework Safely

14.12 Part XII — Building a MiniGateway Application

- 37. Building MiniGateway
- 38. Extending, Testing, and Deploying MiniGateway

14.13 Epilogue

What to Take Away from SNode.C

Chapter 15

Companion source material

The package contains compact source trees for selected HTTP-upgrade, SSE, WebSocket echo server/client subprotocol, MQTT-role, and MariaDB examples. These examples keep the printed chapter fragments focused while giving readers complete CMake-based source directories that can be built against an installed SNode.C package.

The final project source trees are:

```
examples/MiniGateway-Base
examples/MiniGateway-Extended
```

MiniGateway-Base is the source of truth for Chapter 37. **MiniGateway-Extended** is the source of truth for Chapter 38. If a chapter listing and its corresponding example source tree disagree, the example source tree is authoritative and the chapter should be corrected.

Chapter 16

Verification status

The package records author-confirmed local verification for the companion examples and MiniGateway source trees. The verification targets the public SNode.C v1.0.2 tag and exact pinned commit.

The package does not claim independent continuous-integration verification. That is intentional and stated plainly. Independent CI would strengthen a publisher package, but the current manuscript does not present local author verification as a substitute for publisher-side or external CI evidence.

Chapter 17

Evidence and adoption status

The attached evidence sheet summarizes what is publicly verifiable, what is author-confirmed, and what is not yet claimed. The current evidence supports a specialist/project-authoritative publishing case rather than a broad mainstream trade case.

Important facts:

- SNode.C and MQTTSuite are public GitHub repositories.
- MQTTSuite is an applied application ecosystem built on SNode.C.
- The book is tied to a public source baseline.
- The framework has a teaching origin at FH Upper Austria / Campus Hagenberg.
- The current package does not claim download numbers, commercial users, external adoption statistics, or independent course adoptions.

Chapter 18

Commercial / academic fit

The strongest realistic channels are:

- official SNode.C guide;
- specialist professional C++/network-programming title;
- project-authoritative reference;
- course companion for advanced networking, IoT, embedded Linux, media technology, interdisciplinary data-collection, or distributed-systems teaching;
- print-on-demand or community-supported technical book.

A standard broad professional-trade acquisition is harder because the named framework has a capped market unless adoption evidence grows. The proposal therefore should not exaggerate market size. Its best case is depth, completeness, transparency, and usefulness for a defined specialist audience.

Chapter 19

Principal acquisition risks

19.1 Risk 1: Capped market

SNode.C is not a widely known mainstream C++ networking library. The book's acquisition case depends on specialist value, teaching value, and the framework's applied ecosystem rather than broad market familiarity.

19.2 Risk 2: Manuscript length

The manuscript is substantial. Its length is partly justified by the layered teaching arc and MiniGateway capstone, but a standard trade publisher may still require structural compression.

19.3 Risk 3: First-party perspective

The book is written by the framework author. The manuscript handles this through an explicit author/framework disclosure and by avoiding claims of independent comparison.

19.4 Risk 4: Verification status

Verification is author-confirmed local verification, not independent CI. This is disclosed and can be strengthened later by adding public CI around the companion examples.

19.5 Risk 5: Adoption evidence

Current public adoption evidence is modest. The package should not invent users, downloads, commercial deployments, or external course adoptions. If acquisition is the goal, stronger evidence should be gathered separately.

Chapter 20

Risk mitigation already present in the package

- public source-version baseline;
- exact commit pin;
- complete front matter and back matter;
- further reading;
- technical index;
- companion examples;
- MiniGateway base and extended examples;
- explicit author/framework disclosure;
- transparent verification notes;
- proposal evidence sheet.

Chapter 21

Best acquisition argument

This is not a generic C++ networking book. It is a deep, first-party, architecture-first guide to a real layered C++ network framework, with a complete applied MQTT ecosystem and a guided multi-protocol capstone. Its value is strongest where readers need to understand framework design, network-layer boundaries, protocol composition, and maintainable event-driven application structure.

Chapter 22

Recommended publisher handling

If submitted to a standard broad professional publisher, the proposal should be accompanied by adoption evidence and may require a structural length reduction. If submitted to a specialist, academic, course, community, or project-authoritative channel, the current full-length manuscript is much closer to usable because completeness, architecture, examples, and index matter more than mass-market reach.

Contents

Part II

Evidence Sheet: SNode.C Book Proposal

Chapter 23

Purpose of this sheet

This sheet separates three kinds of evidence:

1. public evidence that can be checked from repositories or public documentation;
2. author-confirmed professional context;
3. adoption or market evidence that is not yet claimed.

It is deliberately conservative. The proposal should not invent users, downloads, commercial deployments, course adoptions, or independent validation that has not been documented.

Chapter 24

Public project evidence

24.1 SNode.C repository

- Public repository: SNodeC/snode.c
- Repository visibility: public
- Default branch: **master**
- Manuscript baseline: public release tag **v1.0.2**, commit **6e475262084ae2dab2daef8781ab9e4adb82d18e**
- The repository README describes SNode.C as a lightweight, highly extensible, event-driven, layer-based C++ framework for network applications.
- The README states that development began during the first Corona lockdown in Austria in summer semester 2020 as part of the course *Network and Distributed Systems* in the master's program *Interactive Media* at FH Upper Austria, Campus Hagenberg.
- The README names MQTTSuite as the SNode.C reference project.

24.2 MQTTSuite repository

- Public repository: SNodeC/mqttsuite
- Repository visibility: public
- Default branch: **master**
- The README describes MQTTSuite as a lightweight MQTT 3.1.1 integration system composed of focused applications.
- The listed application roles include MQTTBroker, MQTTIntegrator, MQTTBridge, MQTTCli, and MQTTStore.
- The README states that MQTTSuite is powered by SNode.C.

24.3 Related public repositories

The SNodeC GitHub organization also contains related public repositories that support the ecosystem and book context, including:

- SNodeC/SNode.C-Book
- SNodeC/MiniGateway
- SNodeC/IOTApplicationDesigner
- SNodeC/INJA-Templatebuilder
- SNodeC/CLI11

These repositories demonstrate an ecosystem around the framework and book, but they should not be presented as external adoption evidence unless independent use is documented.

Chapter 25

Applied-system evidence

25.1 MQTTSuite as reference ecosystem

MQTTSuite gives the book a concrete applied context beyond the framework itself. It shows SNode.C used for MQTT-facing applications, including broker, integration, bridge, CLI, and persistence roles. This supports the book's claim that SNode.C is not only a small teaching abstraction but also a basis for multi-application protocol systems.

25.2 MiniGateway as guided capstone

The book package contains two MiniGateway source trees:

```
examples/MiniGateway-Base  
examples/MiniGateway-Extended
```

They support Chapters 37 and 38 and demonstrate a small SNode.C application that combines HTTP status routes, SSE observation, MQTT publication, application-owned state, and an added Unix-domain input boundary in the extended version.

MiniGateway is evidence of pedagogical completeness and architectural applicability inside the manuscript package. It is not presented as external market adoption.

25.3 Companion examples

The package also contains companion source trees for selected protocol and persistence examples, including HTTP upgrade, SSE, WebSocket echo subprotocol examples, MQTT client role, and MariaDB. These examples support the reader promise and reduce the risk that the manuscript is only prose without buildable companion material.

Chapter 26

Teaching-context evidence

The SNode.C README publicly ties the framework’s origin to a teaching context: the *Network and Distributed Systems* course in the master’s program *Interactive Media* at FH Upper Austria, Campus Hagenberg.

This supports the book’s educational positioning, but it should not be overstated as independent course adoption of the finished book unless such adoption is documented separately.

Chapter 27

Educational and interdisciplinary technical use cases

The manuscript explicitly supports educational and interdisciplinary technical use cases, including networked measurement, environmental monitoring, field sensing, lab instrumentation, gateway-style integration systems, and technical data-collection projects. These are presented as architectural use cases for readers who need to understand, build, teach, specify, adapt, or supervise such systems, not as claims of external adoption.

Chapter 28

Author-confirmed professional context

The following background items are author-confirmed and relevant to credibility, but should be treated as author-supplied CV context unless independent public sources are attached later:

- studies in theoretical physics in Graz;
- work at Ars Electronica Futurelab;
- four years as university assistant at Johannes Kepler University Linz in the Pervasive Computing group;
- work with ORF Kunstradio;
- participation around documenta X in Kassel;
- many projects connected to Ars Electronica Futurelab and media technology.

This background supports the author's technical and media-technology authority, but the proposal should distinguish it from repository-based evidence.

Chapter 29

Verification evidence inside the package

The package includes verification notes for the examples and MiniGateway source trees:

```
verification/examples-aggregate-build-verification.md
verification/minigateway-step8-author-verification.md
verification/book-source-baseline.md
verification/book-source-baseline.env
```

These documents record author-confirmed local verification against the public SNode.C source baseline. They are useful package evidence, but they are not independent CI evidence.

Chapter 30

What is not yet claimed

The current proposal does not claim:

- commercial deployments;
- external customers;
- download statistics;
- package-manager install counts;
- GitHub stars/forks as market proof;
- third-party dependent projects;
- independent course adoption of the finished book;
- independent CI verification of all companion examples.

This restraint is intentional. Unsupported adoption claims would weaken the proposal.

Chapter 31

Evidence gaps to close for a stronger acquisition case

A broader professional-trade acquisition case would be stronger with:

- documented public release/download statistics;
- list of known users or organizations, if any can be named;
- dependent repositories or downstream projects;
- course-use statement or syllabus reference;
- independent reviewer quotes;
- CI badge or reproducible build workflow for companion examples;
- public package/deployment evidence for Linux or OpenWrt if available;
- a concise statement of why readers who do not yet use SNode.C should care.

Chapter 32

Current evidence verdict

The evidence currently supports a specialist, project-authoritative, course-friendly, or community technical-book case. It does not yet support an exaggerated mass-market claim. The proposal should therefore sell the book on depth, completeness, architecture, source alignment, examples, transparency, and the applied MQTTSuite ecosystem.